6.4 Connect the product route and view

Finish the product feature by connecting its view, navigation link, and route. Keep Product and ProductController unchanged from 6.3.

Add the product view

Create `templates/tutorials/class6/products.html.twig`:

```
{% extends 'base.html.twig' %}
{% block pageTitle %}Products{% endblock %}
{% block productsLink %}active{% endblock %}
{% block main %}
    <p>Supplies for your study space.</p>
    <dl>
        {% for product in products %}
            <dt>{{ product.name }}</dt>
            <dd>${{ product.price|number_format(2) }}</dd>
        {% else %}
            <dt>Availability</dt>
            <dd>No products available.</dd>
        {% endfor %}
    </dl>
{% endblock %}
```

The products context key becomes the template's products variable. Each loop iteration reads one object's public name and price. number_format controls presentation. The loop's else branch handles an empty collection; it does not handle a missing context key with strict variables enabled.

Add shared navigation

In `templates/tutorials/class6/base.html.twig`, add this link inside nav, immediately after the Contact link:

```
<a class="{% block productsLink %}{% endblock %}" href="index.php?
action=products">Products</a>
```

Because all views extend base, the new link appears on all three pages. Only the Products child overrides productsLink with active.

Replace Application with the three-page router

Replace `src/Tutorials/Class6/Application.php` with:

```php
<?php
declare(strict_types=1);

namespace App\Tutorials\Class6;

class Application
{
    public function run(): void
    {
        $defaultController = new DefaultController();
        $productController = new ProductController();

        $action = $_GET['action'] ?? 'home';
        if (!is_string($action)) {
            $action = 'home';
        }

        switch ($action) {
            case 'products':
                $productController->productList();
                break;
            case 'contact':
                $defaultController->contact();
                break;
            case 'home':
            default:
                $defaultController->home();
        }
    }
}
```

The action string products, method name productList, and filename products.html.twig are connected explicitly by our code. PHP does not infer the relationship from the names. We create both controller objects near the start of run, as the chapter does. Constructing them initializes Twig; only the selected action renders a page.

Verify the complete feature

1. Open `?action=products`. Expect Study notebook / $4.50, Desk organizer / $12.00, and Pencil set / $3.25.
2. Follow all three navigation links. Each destination should have the correct heading and exactly one active link.
3. Temporarily replace the products array in productList with an empty array. Reload Products and expect “No products available.” Restore the three objects afterward.
4. Try `?action=unknown` and `?action[]=products`; both should still show Home.